

Лабораторная работа 10.

Создание макета страницы и верстка

Создание макета страницы и верстка. Блочная верстка

Как правило, веб-страница состоит из множества различных элементов, которые могут иметь сложную структуру. Поэтому при создании веб-страницы возникает необходимость нужным образом позиционировать эти элементы, стилизовать их так, чтобы они располагались на странице нужным образом. То есть возникает вопрос создания макета страницы, ее верстки.

Существуют различные способы, стратегии и виды верстки. Изначально распространенной была верстка на основе таблиц. Так как таблицы позволяют при необходимости очень легко и просто разделить все пространство веб-страницы на строки и столбцы. Строками и столбцами довольно легко управлять, в них легко позиционировать любое содержимое. Именно это и определило популярность табличной верстки.

Однако табличная верстка создает не самые гибкие по дизайну страницы, что является особенно актуальным аспектом в мире, где нет одного единственного разрешения экрана, за то есть большие экраны на телевизорах, малые экраны на планшетах и факлетах, очень маленькие экраны на смартфонах и т.д. Все это многообразие экранов табличная верстка оказалась не в состоянии удовлетворить. Поэтому постепенно ей на смену пришла блочная верстка. Блочная верстка - это относительно условное название способов и приемов верстки, когда в большинстве веб-страниц для разметки используется CSS-свойство `float`, а основным строительным элементом веб-страниц является элемент `<div>`, то есть по сути блок. Используя свойство `float` и элементы `div` или другие элементы, можно создать структуру страницы из нескольких столбцов, как при табличной верстке, которая будет значительно гибче.

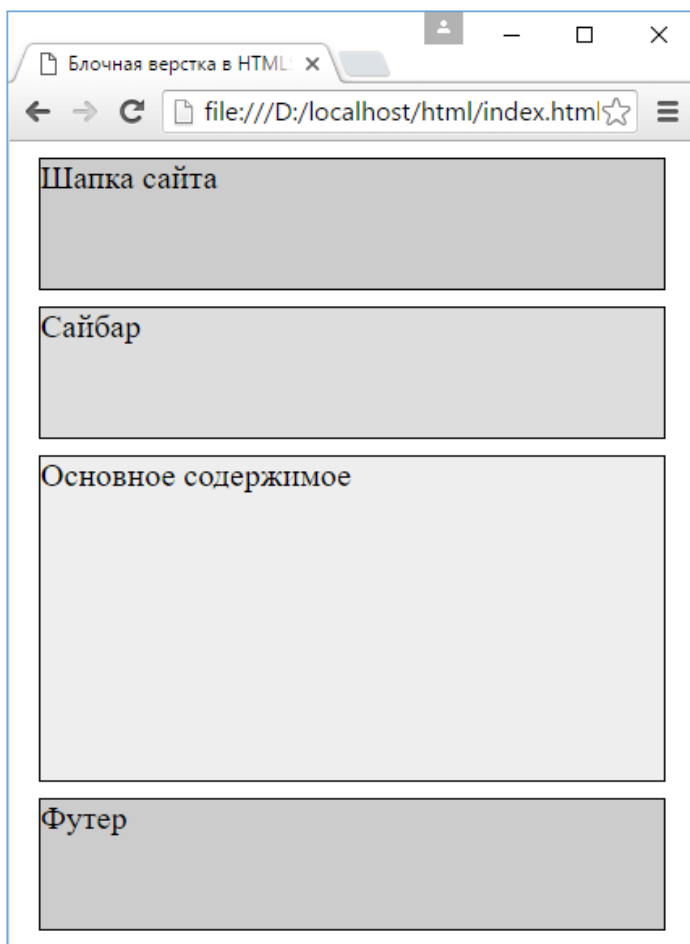
Ранее в одной из прошлых тем рассматривалось действие свойства `float`. Теперь используем его для создания двухколоночной веб-страницы. Допустим, вверху и внизу у нас будут стандартно шапка и футер, а в центре - две колонки: колонка с меню или сайдбар и колонка с основным содержимым.

В начале определим все блоки. При работе с элементами, которые используют обтекание и свойство `float`, важен их порядок. Так, код плавающего элемента, у которого устанавливается свойство `float`, должен идти перед элементом, который обтекает плавающий элемент. То есть блок сайдбара будет идти до блока основного содержимого:

Пример 1.

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <title>Блочная верстка в HTML5</title>
    <style>
      div{
        margin: 10px;
        border: 1px solid black;
        font-size: 20px;
        height: 80px;
      }
      #header{
        background-color: #ccc;
      }
      #sidebar{
        background-color: #ddd;
      }
      #main{
        background-color: #eee;
        height: 200px;
      }
      #footer{
        background-color: #ccc;
      }
    </style>
  </head>
  <body>
    <div id="header">Шапка сайта</div>
    <div id="sidebar">Сайдбар</div>
    <div id="main">Основное содержимое</div>
    <div id="footer">Футер</div>
  </body>
</html>
```

То есть пока получается примерно следующая страница:



Высота, граница и отступы блоков в данном случае добавлены только для красоты, чтобы идентифицировать пространство блока и отделять его от других.

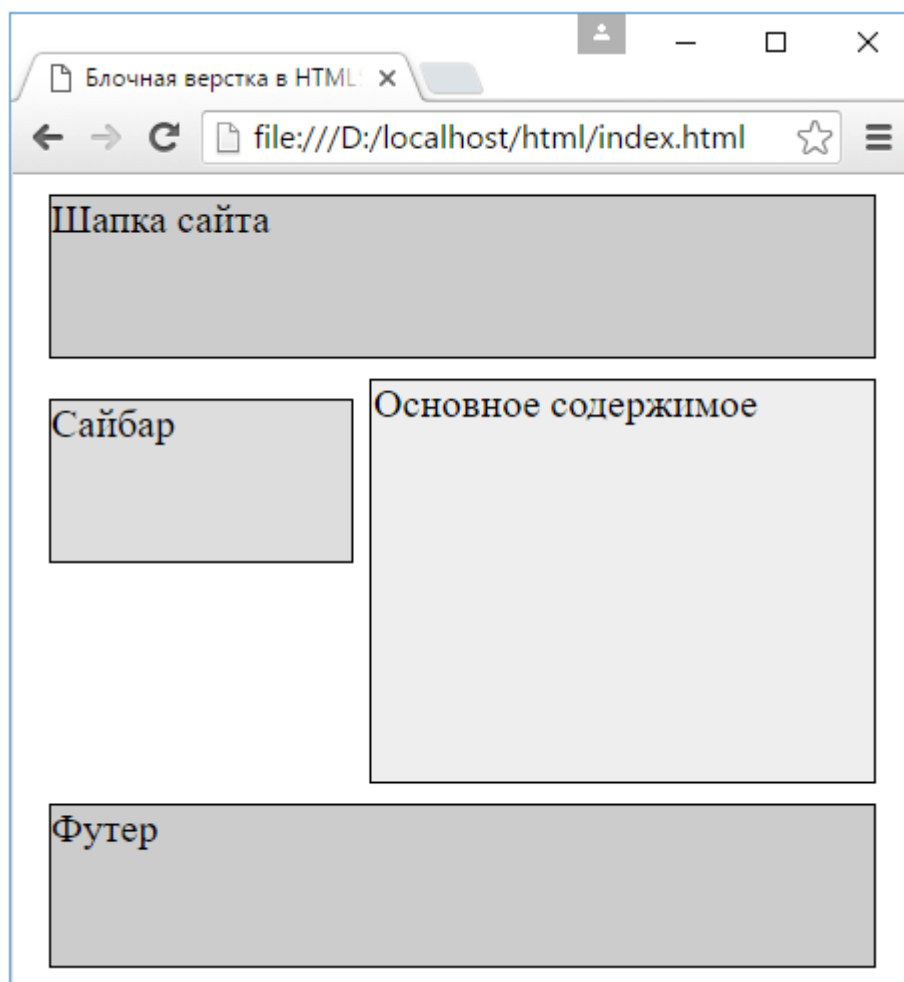
Далее, чтобы переместить блок сайдбара влево по отношению к блоку основного содержимого и получить эффект обтекания, нам надо указать у блока сайдбара свойство `float: left` и предпочтительную ширину. Ширина может быть фиксированной, например, `150px` или `8em`. Либо также можно использовать проценты, например, `30%` - 30% от ширины контейнера `body`. С одной стороны, блоками с фиксированной шириной легче управлять, но с другой процентные значения ширины позволяют создавать более гибкие, резиновые блоки, которые изменяют размеры при изменении размеров окна браузера.

Последним шагом является установка отступа блока с основным содержимым от блока сайдбара. Поскольку при обтекании обтекающий блок может обтекать плавающий элемент и справа и снизу, если плавающий элемент имеет меньшую высоту, то нам надо установить отступ, как минимум равный ширине плавающего элемента. Например, если ширина сайдбара равна `150px`, то для блока основного содержимого можно задать отступ в `170px`, что позволит создать пустое пространство между двумя блоками.

При этом не стоит у блока основного содержимого указывать явным образом ширину, так как браузеры расширяют его автоматически, чтобы он занимал все доступное место.

Итак, принимая во внимание все выше сказанное, изменим стили блоков сайдбара и основного содержимого следующим образом:

```
#sidebar{
    background-color: #ddd;
    float: left;
    width: 150px;
}
#main{
    background-color: #eee;
    height: 200px;
    margin-left: 170px; /* 150px (ширина сайдбара) + 10px + 10px (2 отступа)
*/
```



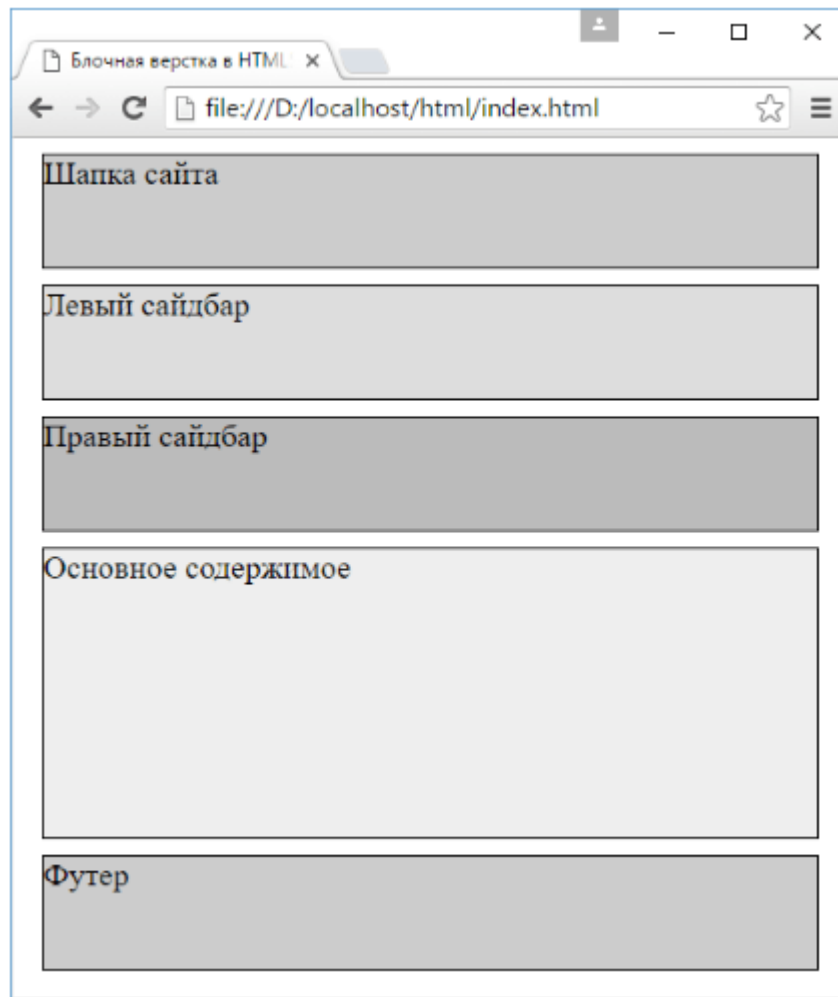
Высота блоков в данном случае указана условно для большей наглядности, в реальности, как правило, высоту будет автоматически устанавливать браузер.

Пример 2.

В прошлом примере было рассмотрено создание страницы с двумя колонками. Подобным образом мы можем добавить на страницу и большее количество колонок и сделать более сложную структуру.

Например, добавим на веб-страницу второй сайдбар:

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <title>Блочная верстка в HTML5</title>
    <style>
      div{
        margin: 10px;
        border: 1px solid black;
        font-size: 20px;
        height: 80px;
      }
      #header{
        background-color: #ccc;
      }
      #leftSidebar{
        background-color: #ddd;
      }
      #rightSidebar{
        background-color: #bbb;
      }
      #main{
        background-color: #eee;
        height: 200px;
      }
      #footer{
        background-color: #ccc;
      }
    </style>
  </head>
  <body>
    <div id="header">Шапка сайта</div>
    <div id="leftSidebar">Левый сайдбар</div>
    <div id="rightSidebar">Правый сайдбар</div>
    <div id="main">Основное содержимое</div>
    <div id="footer">Футер</div>
  </body>
</html>
```

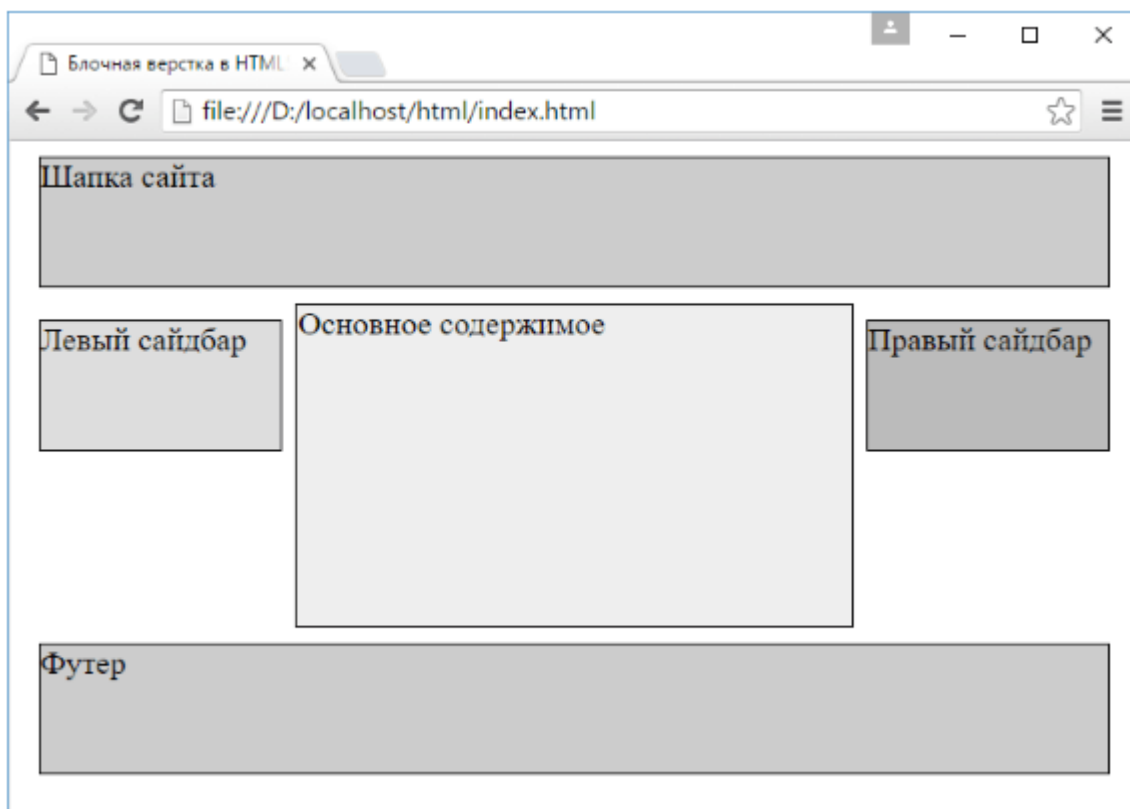


Здесь опять же код обоих сайдбаров должен идти до блока с основным содержимым, который обтекает их.

Теперь изменим стили обоих сайдбаров и основного блока:

```
#leftSidebar{
  background-color: #ddd;
  float: left;
  width: 150px;
}
#rightSidebar{
  background-color: #bbb;
  float: right;
  width: 150px;
}
#main{
  background-color: #eee;
  height: 200px;
  margin-left: 170px;
  margin-right: 170px;
}
```

Опять же у обоих плавающих блоков - сайдбаров нам надо установить ширину и свойство `float` - у одного значение `left`, а у другого `right`.



Вложенные плавающие блоки

Нередко встречается ситуация, когда к вложенным в обтекающий блок элементам также применяется обтекание. Например, блок основного содержимого может включать блок собственно содержимого и блок меню. В принципе к таким блокам будут применяться все те же правила, что были рассмотрены ранее.

Определим сначала последовательно все блоки веб-страницы:

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <title>Блочная верстка в HTML5</title>
    <style>
      div{
        margin: 10px;
        border: 1px solid black;
        font-size: 20px;
        height: 80px;
      }
      #header{
        background-color: #ccc;
      }
      #sidebar{
        background-color: #bbb;
        float: right;

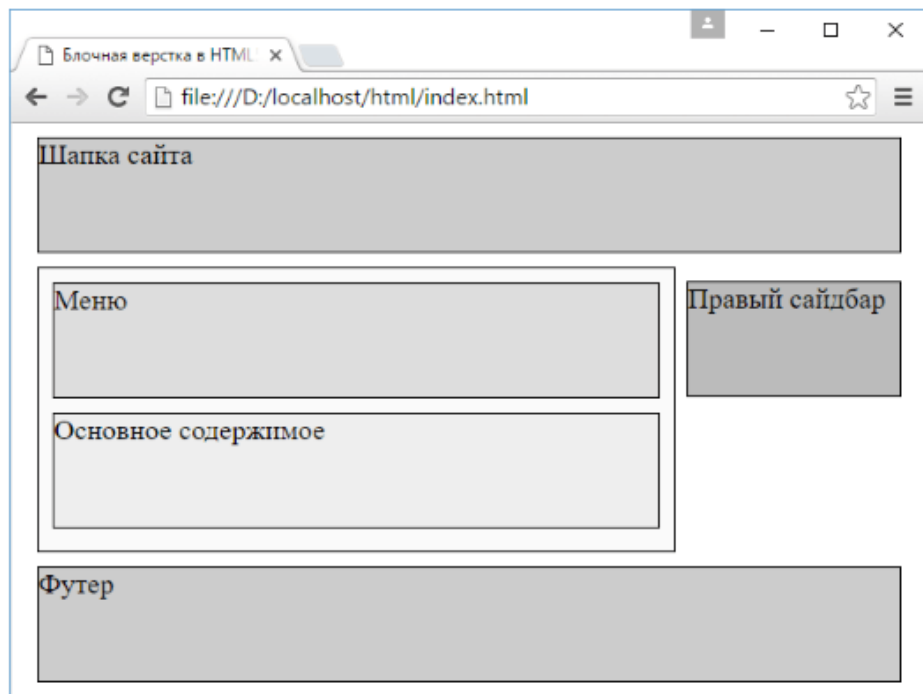
```

```

        width: 150px;
    }
    #main{
        background-color: #fafafa;
        height: 200px;
        margin-right: 170px;
    }
    #menu{
        background-color: #ddd;
    }
    #content{
        background-color: #eee;
    }
    #footer{
        background-color: #ccc;
    }
</style>
</head>
<body>
    <div id="header">Шапка сайта</div>
    <div id="sidebar">Правый сайдбар</div>
    <div id="main">
        <div id="menu">Меню</div>
        <div id="content">Основное содержимое</div>
    </div>
    <div id="footer">Футер</div>
</body>
</html>

```

Опять же в главном блоке вложенные блоки идут последовательно: сначала блок меню, а потом блок основного текста.



Теперь применим обтекание к блоку меню:

```

#menu{
    background-color: #ddd;

```

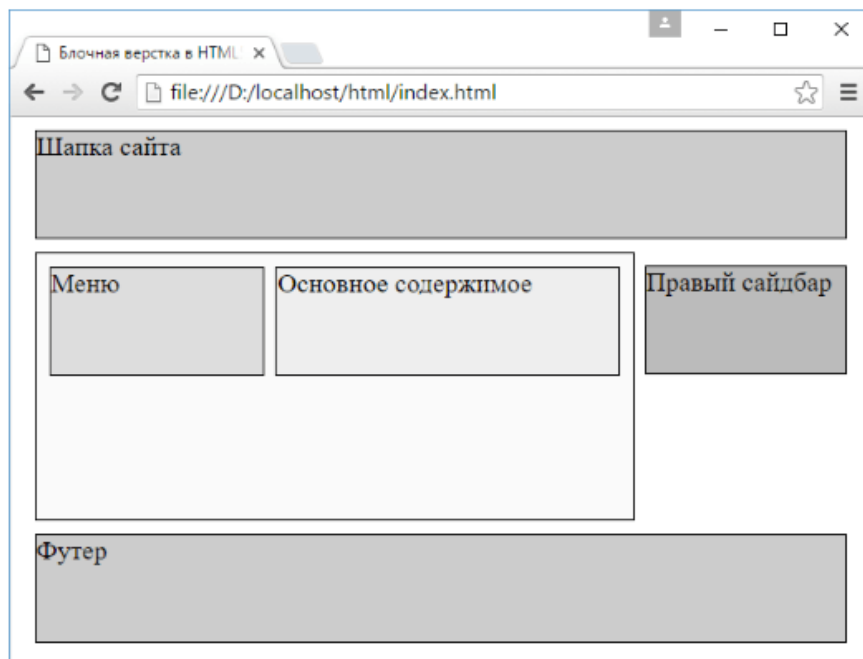


```

float: left;
width: 160px;
}
#content{
background-color: #eee;
margin-left: 180px;
}

```

Опять же у плавающего элемента, коим является блок меню, устанавливаются свойства float и width. А у обтекающего его блока content устанавливается отступ слева.



Аналогично можно сделать блок меню справа:

```

#menu{
background-color: #ddd;
float: right;
width: 160px;
}
#content{
background-color: #eee;
margin-right: 180px;
}

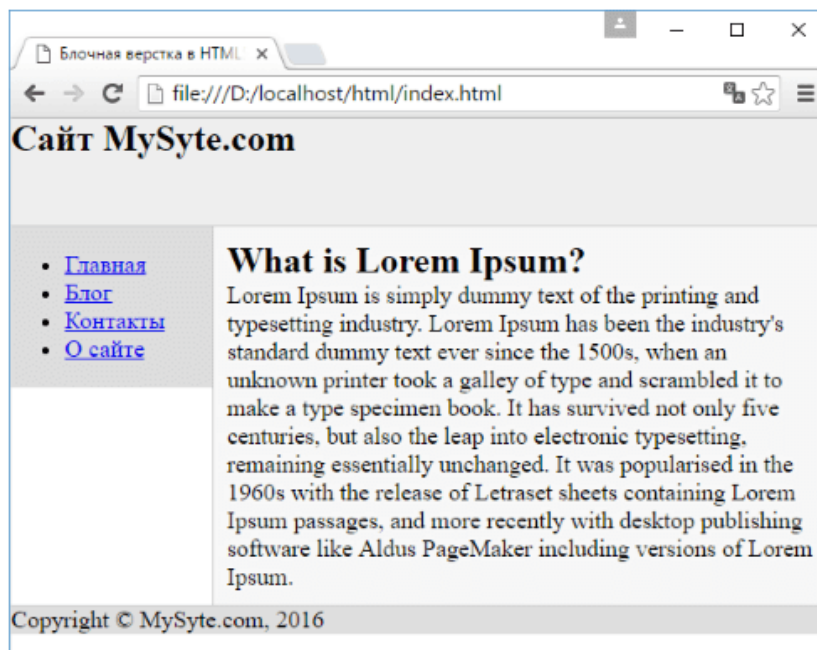
```

Выравнивание столбцов по высоте

При блочной верстке мы можем столкнуться с проблемой высоты с столбцов, которая может особенно сильно проявиться, если плавающие блоки имеют определенный фон. Рассмотрим проблему на примере:

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <title>Блочная верстка в HTML5</title>
    <style>
      body, h2, p{
        margin:0;
        padding:0;
      }
      body{
        font-size: 18px;
      }
      #header{
        background-color: #eee;
        border-bottom: 1px solid #ccc;
        height: 80px;
      }
      #menu{
        background-color: #ddd;
        float: left;
        width: 150px;
      }
      #main{
        background-color: #f7f7f7;
        border-left: 1px solid #ccc;
        margin-left: 150px;
        padding: 10px;
      }
      #footer{
        border-top: 1px solid #ccc;
        background-color: #dedede;
      }
    </style>
  </head>
  <body>
    <div id="header"><h2>Сайт MySyte.com</h2></div>
    <div id="menu">
      <ul>
        <li><a href="#">Главная</a></li>
        <li><a href="#">Блог</a></li>
        <li><a href="#">Контакты</a></li>
        <li><a href="#">О сайте</a></li>
      </ul>
    </div>
    <div id="main">
      <h2>What is Lorem Ipsum?</h2>
      <p>Lorem Ipsum is simply dummy text of the printing and
typesetting industry. Lorem Ipsum
      has been the industry...</p>
    </div>
    <div id="footer">
      <p>Copyright © MySyte.com, 2016</p>
    </div>
  </body>
</html>
```

Здесь уже установлено обтекание, и оно прекрасно работает, вот только в зависимости от количества содержимого одна колонка может быть больше другой:



В данном случае, если в главном блоке много текста, то блок с меню имеет недостаточную длину.

Наиболее распространенным решением данной проблемы является оборачивание плавающего элемента и блока, его обтекающего, в отдельный элемент, у которого устанавливается фон. Затем этот фон используется наименьшим по длине блоком. В итоге получается иллюзия, что блоки имеют равную длину, а фон у блоков установлен корректно.

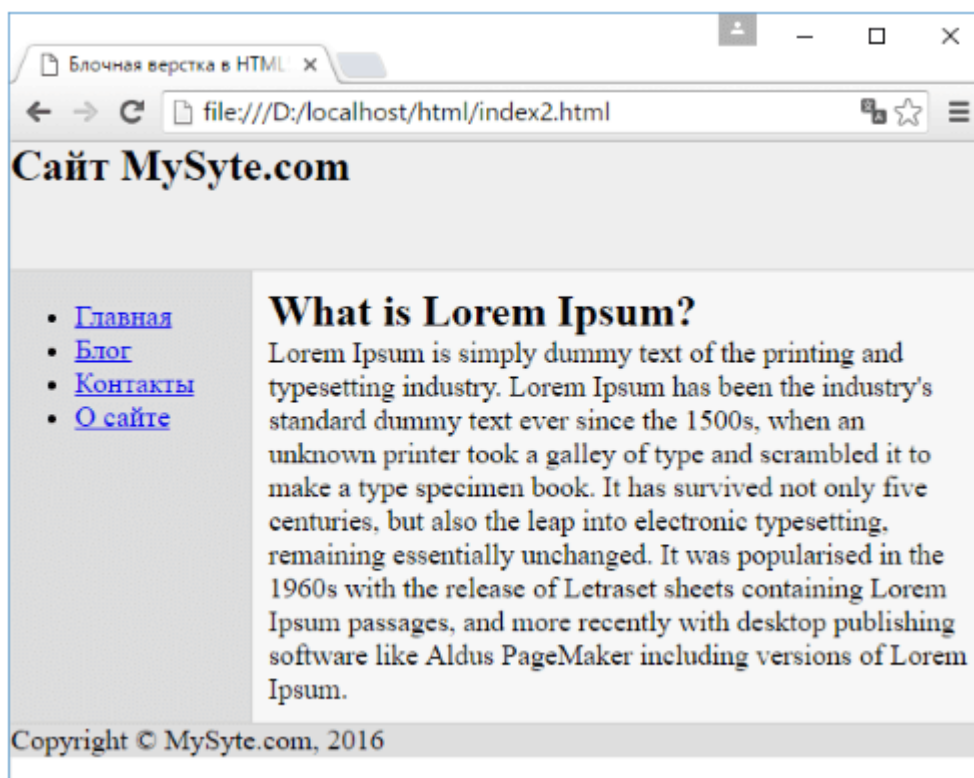
Так, изменим выше созданную страницу следующим образом:

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <title>Блочная верстка в HTML5</title>
    <style>
      body, h2, p{
        margin:0;
        padding:0;
      }
      body{
        font-size: 18px;
      }
      #header{
        background-color: #eee;
        border-bottom: 1px solid #ccc;
        height: 80px;
      }
      #wrapper{
```

```

        background-color: #ddd;
    }
    #menu{
        float: left;
        width: 150px;
    }
    #main{
        background-color: #f7f7f7;
        border-left: 1px solid #ccc;
        margin-left: 150px;
        padding: 10px;
    }
    #footer{
        border-top: 1px solid #ccc;
        background-color: #dedede;
    }
</style>
</head>
<body>
    <div id="header"><h2>Сайт MySyte.com</h2></div>
    <div id="wrapper">
        <div id="menu">
            <ul>
                <li><a href="#">Главная</a></li>
                <li><a href="#">Блог</a></li>
                <li><a href="#">Контакты</a></li>
                <li><a href="#">0 сайте</a></li>
            </ul>
        </div>
        <div id="main">
            <h2>What is Lorem Ipsum?</h2>
            <p>Lorem Ipsum is simply dummy text of the printing and
typesetting industry. Lorem Ipsum
                has been the industry...</p>
        </div>
    </div>
    <div id="footer">
        <p>Copyright © MySyte.com, 2016</p>
    </div>
</body>
</html>

```



В данном случае плавающий блок menu и обтекающий его блок main обертываются в один элемент wrapper, в котором устанавливается фон для элемента menu. А элемент main, как наибольший элемент, может использовать собственную установку фона.

Свойство display

Кроме свойства `float`, которое позволяет изменять позицию элемента, в CSS есть еще одно важное свойство - `display`. Оно позволяет управлять блоком элемента и также влиять на его позиционирование относительно соседних элементов.

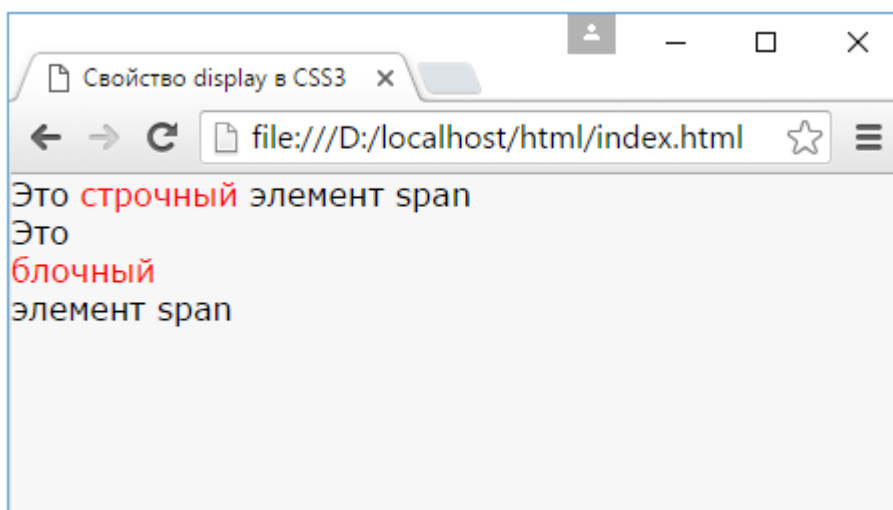
Это свойство может принимать следующие значения:

- `inline`: элемент становится строчным, подобно словам в строке текста
- `block`: элемент становится блочным, как параграф
- `inline-block`: элемент располагается как строка текста
- `list-item`: элемент позиционируется как элемент списка обычно с добавлением маркера в виде точки или порядкового номера
- `run-in`: тип блока элемента зависит от окружающих элементов
- `flex`: позволяет осуществлять гибкое позиционирование элементов
- `table`, `inline-table`: позволяет расположить элементы в виде таблицы
- `none`: элемент не виден и удален из разметки html

Итак, значение `block` позволяет определить блочный элемент. Такой элемент визуально отделяется от соседних элементов переносом строки, как, например, элемент параграфа `p` или элемент `div`, которые по умолчанию являются блочными и при визуализации веб-страницы визуально переносятся на новую строку.

Однако элемент `span` в отличие от элемента `div` блочным не является. Поэтому посмотрим, какие с ним произойдут изменения при применении значения `block`:

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <link href="styles.css" rel="stylesheet">
    <title>Свойство display в CSS3</title>
    <style>
      span{
        color: red;
      }
      .blockSpan{
        display: block;
      }
    </style>
  </head>
  <body>
    <div>Это <span>строчный</span> элемент span</div>
    <div>Это <span class="blockSpan">блочный</span> элемент span</div>
  </body>
</html>
```

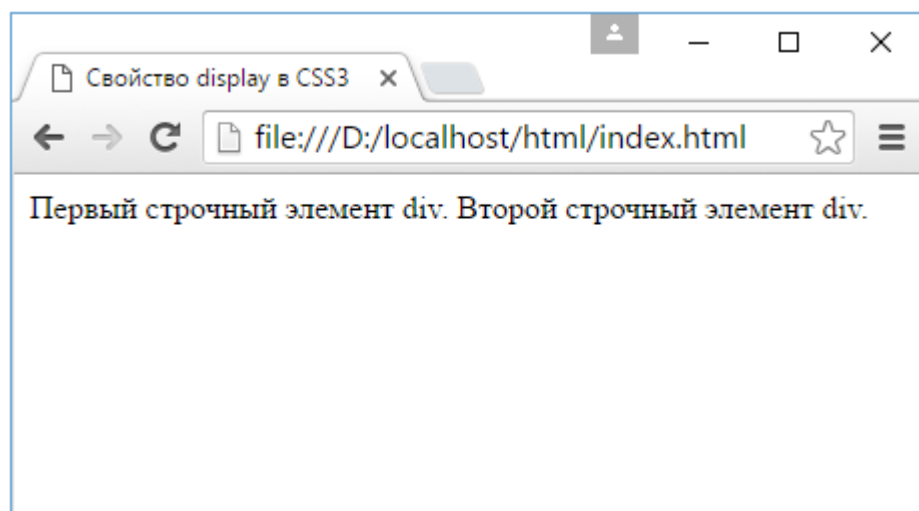


Здесь определено два элемента `span`, но один из них является блочным, так как к нему применяется стиль `display: block`. Поэтому этот элемент `span` переносится на новую строку.

В отличие от блочных элементов строчные встраиваются в строку, так как имеют для свойства `display` значение `inline`. Элемент `span` как раз по умолчанию имеет стиль `display: inline`, поэтому и встраивается в строку, а не переносится на следующую,

как параграфы или `div`. И теперь произведем обратную процедуру - сделаем блочный элемент `div` строчным:

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <title>Свойство display в CSS3</title>
    <style>
      div{
        display: inline;
      }
    </style>
  </head>
  <body>
    <div>Первый строчный элемент div.</div>
    <div>Второй строчный элемент div.</div>
  </body>
</html>
```



Следует учитывать, что при применении значения `inline` браузер игнорирует некоторые свойства, такие как `width`, `height`, `margin`.

inline-block

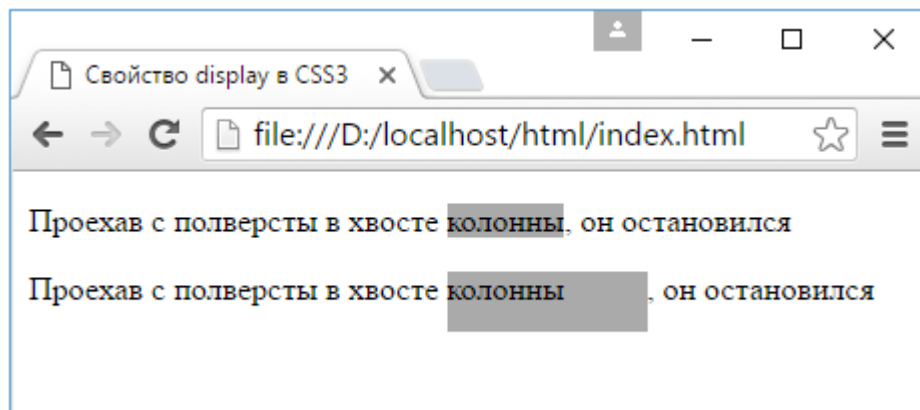
Еще одно значение - `inline-block` - представляет элемент, который обладает смесью признаков блочного и строчного элементов. По отношению к соседним внешним элементам такой элемент расценивается как строчный. То есть он не отделяется от соседних элементов переводом строки. Однако по отношению к вложенным элементам он рассматривается как блочный. И к такому элементу применяются свойства `width`, `height`, `margin`.

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <title>Свойство display в CSS3</title>
    <style>
      span{
        width: 100px;
        height: 30px;
      }
    </style>
  </head>
  <body>
    <span>Первый строчный элемент span.</span>
    <span>Второй строчный элемент span.</span>
  </body>
</html>
```

```

        background-color: #aaa;
    }
    .inlineBlockSpan{
        display: inline-block;
    }
</style>
</head>
<body>
    <p>Проехав с полверсты в хвосте <span>колонны</span>, он
остановился</p>
    <p>Проехав с полверсты в хвосте <span
class="inlineBlockSpan">колонны</span>, он остановился</p>
</body>
</html>

```



Первый элемент `span` является строчным, у него значение `inline`, поэтому для него бессмысленно применять свойства `width` и `height`. А вот второй элемент `span` имеет значение `inline-block`, поэтому к нему уже применяются и ширина, и высота, и при необходимости еще можно установить отступы.

run-in

Значение `run-in` определяет элемент, который зависит от соседних элементов. И здесь есть три возможных варианта:

- Элемент окружен блочными элементами, тогда фактически он имеет стиль `display: block`, то есть сам становится блочным
- Элемент окружен строчными элементами, тогда фактически он имеет стиль `display: inline`, то есть сам становится строчным
- Во всех остальных случаях элемент считается блочным

Табличное представление

Значение `table` по сути превращает элемент в таблицу. Посмотрим на примере списка:

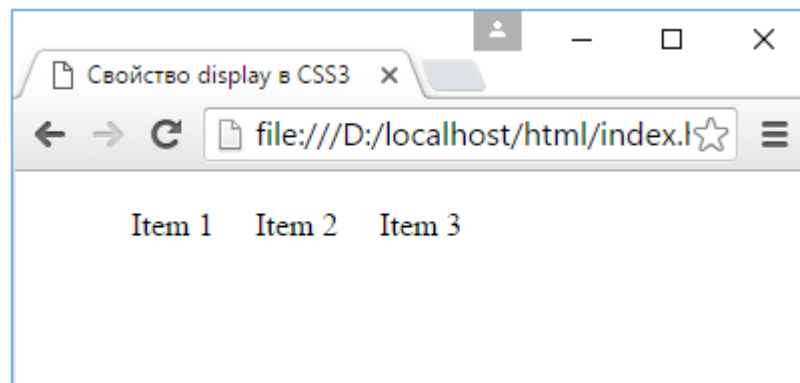
```
<!DOCTYPE html>
```



```

<html>
  <head>
    <meta charset="utf-8">
    <title>Свойство display в CSS3</title>
    <style>
      ul{
        display: table;
        margin: 0;
      }
      li{
        list-style-type: none;
        display: table-cell;
        padding: 10px;
      }
    </style>
  </head>
  <body>
    <ul>
      <li>Item 1</li>
      <li>Item 2</li>
      <li>Item 3</li>
    </ul>
  </body>
</html>

```



Здесь список превращается в таблицу, а каждый элемент списка - в отдельную ячейку. Для этого у элемента списка устанавливается стиль `display: table-cell`. Фактически вместо этого списка мы могли бы использовать стандартную таблицу.

Соккрытие элемента

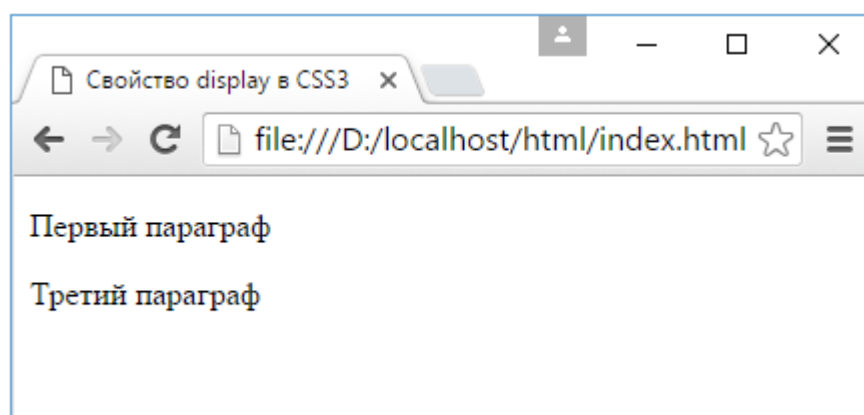
Значение `display: none` позволяет скрыть элемент, которого как-будто нет на веб-странице:

```

<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <title>Свойство display в CSS3</title>
    <style>
      .invisible{
        display: none;
      }
    </style>
  </head>
  <body>
    <p>Первый параграф</p>
    <p class="invisible">Второй параграф</p>
    <p>Третий параграф</p>
  </body>
</html>

```

```
</body>  
</html>
```



Создание панели навигации

Панель навигации играет важную роль на сайте, так как обеспечивает переходы между страницами сайта или на внешние ресурсы. Рассмотрим, как создать простенькую панель навигации.

Фактически панель навигации — это набор ссылок, часто в виде нумерованного списка. Панели навигации бывают самыми различными: вертикальными и горизонтальными, одноуровневыми и многоуровневыми, но в любом случае в центре каждой навигации находится элемент `<a>`. Поэтому при создании панели навигации мы можем столкнуться с рядом трудностей, которые вытекают из ограничений элемента ссылки. А именно, элемент `<a>` является строчным, а это значит, что мы не можем указать для него ширину, высоту, отступы. По ширине ссылка автоматически занимает то место, которое ей необходимо.

Вертикальное меню

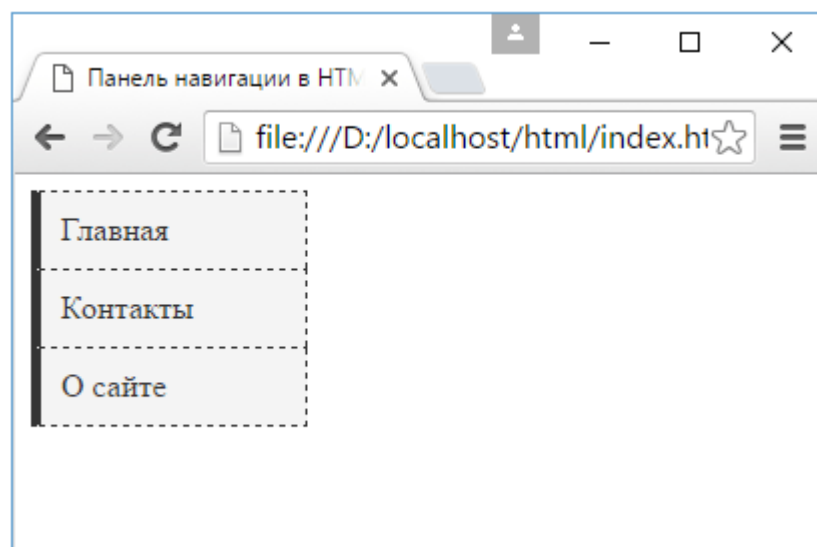
Распространенное решение данной проблемы для создания вертикального меню состоит в том, чтобы сделать ссылку блочным элементом.

```
<!DOCTYPE html>  
<html>  
  <head>  
    <meta charset="utf-8">  
    <title>Панель навигации в HTML5</title>  
    <style>  
      ul.nav{  
        margin-left: 0px;  
        padding-left: 0px;
```

```

        list-style: none;
    }
    ul.nav a {
        display: block;
        width: 7em;
        padding: 10px;
        background-color: #f4f4f4;
        border-top: 1px dashed #333;
        border-right: 1px dashed #333;
        border-left: 5px solid #333;
        text-decoration: none;
        color: #333;
    }
    ul.nav li:last-child a {
        border-bottom: 1px dashed #333;
    }
</style>
</head>
<body>
    <ul class="nav">
        <li><a href="#">Главная</a></li>
        <li><a href="#">Контакты</a></li>
        <li><a href="#">О сайте</a></li>
    </ul>
</body>
</html>

```



После установки свойства `display: block` мы можем определить у блока ссылки ширину, отступы и т.д.

Горизонтальное меню

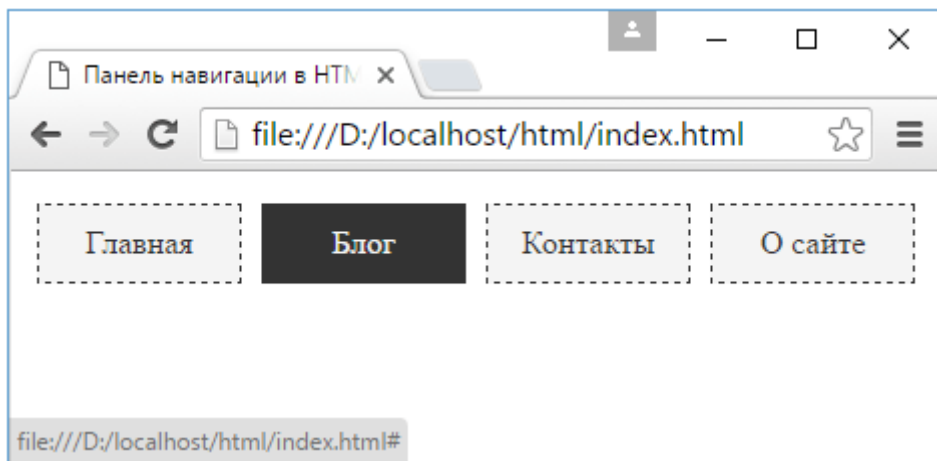
Для создания горизонтального меню есть два метода. Первый заключается в применении свойства `float` и создании из ссылок плавающих элементов, которые обтекают друг друга с слева. И второй способ состоит в создании строки ссылок с помощью установки свойства `display: inline-block`.

Использование float

Алгоритм создания панели навигации с помощью `float` разделяется на два этапа. На первом этапе у элемента `li`, в который заключена ссылка, устанавливается `float: left;`. Это позволяет расположить все элементы списка в ряд при достаточной ширине, когда правый элемент списка обтекает левый элемент списка.

Второй этап заключается в установке у элемента ссылки `display: block`, что дает нам возможность устанавливать ширину, отступы, вообще все те признаки, которые характерны для блочных элементов.

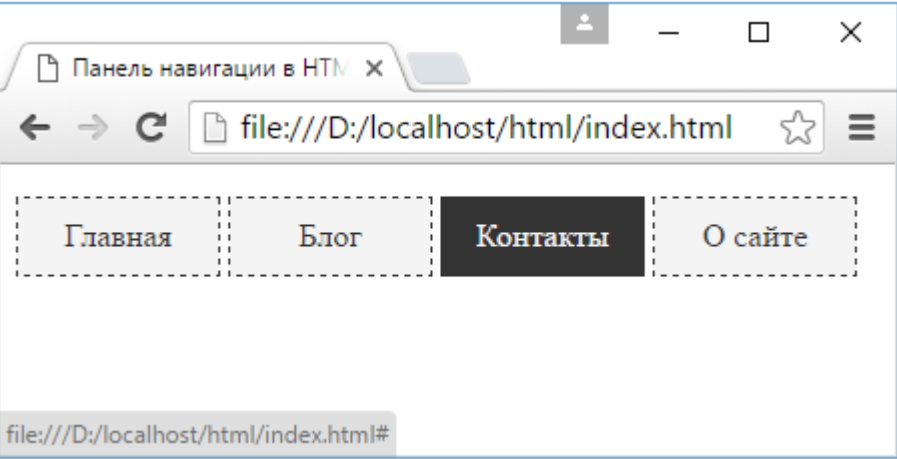
```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <title>Панель навигации в HTML5</title>
    <style>
      ul.nav{
        margin-left: 0px;
        padding-left: 0px;
        list-style: none;
      }
      .nav li {
        float: left;
      }
      ul.nav a {
        display: block;
        width: 5em;
        padding: 10px;
        margin: 0 5px;
        background-color: #f4f4f4;
        border: 1px dashed #333;
        text-decoration: none;
        color: #333;
        text-align: center;
      }
      ul.nav a:hover{
        background-color: #333;
        color: #f4f4f4;
      }
    </style>
  </head>
  <body>
    <ul class="nav">
      <li><a href="#">Главная</a></li>
      <li><a href="#">Блог</a></li>
      <li><a href="#">Контакты</a></li>
      <li><a href="#">О сайте</a></li>
    </ul>
  </body>
</html>
```



inline и inline-block

Для создания горизонтальной панели навигации нам надо сделать каждый элемент `li` строчным, то есть установить для него `display: inline`. После этого для элемента ссылки, которая располагается в элементе `li`, мы можем установить `display: inline-block`:

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <title>Панель навигации в HTML5</title>
    <style>
      ul.nav{
        margin-left: 0px;
        padding-left: 0px;
        list-style: none;
      }
      .nav li {
        display: inline;
      }
      ul.nav a {
        display: inline-block;
        width: 5em;
        padding: 10px;
        background-color: #f4f4f4;
        border: 1px dashed #333;
        text-decoration: none;
        color: #333;
        text-align: center;
      }
      ul.nav a:hover{
        background-color: #333;
        color: #f4f4f4;
      }
    </style>
  </head>
  <body>
    <ul class="nav">
      <li><a href="#">Главная</a></li>
```



Выравнивание плавающих элементов

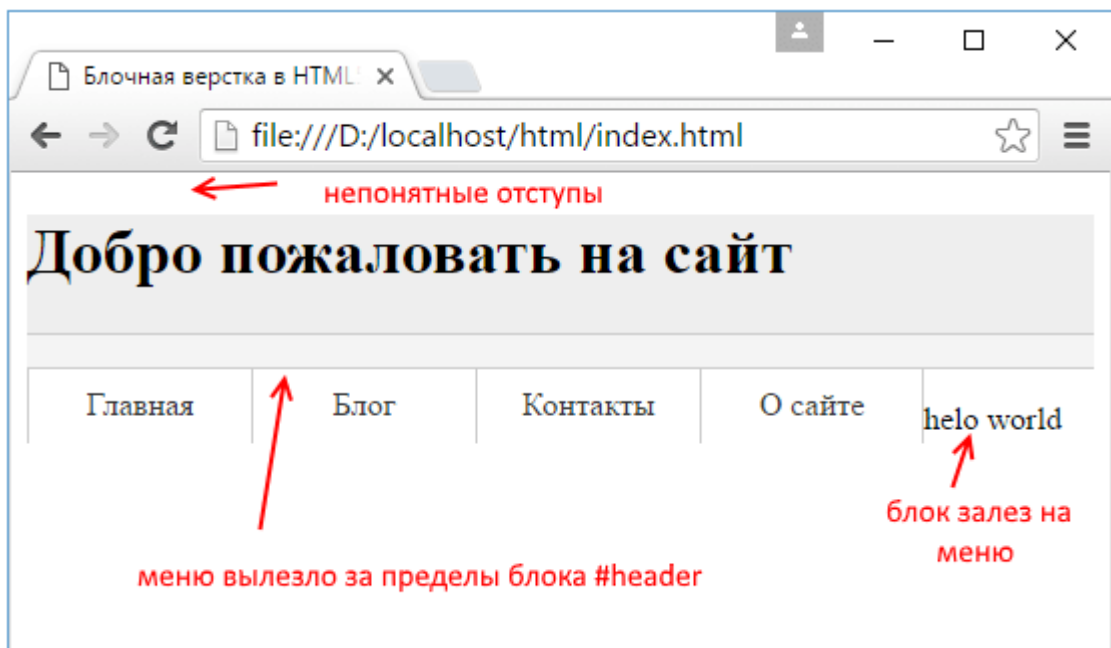
<head>

```

    }
    #nav ul li:last-child a {

        border-right: 1px solid #ccc;
    }
    #nav ul a:hover{
        background-color: #aaa;
        color: #f4f4f4;
    }
}
</style>
</head>
<body>
    <div id="header">
        <h1>Добро пожаловать на сайт</h1>
        <div id="nav">
            <ul>
                <li><a href="#">Главная</a></li>
                <li><a href="#">Блог</a></li>
                <li><a href="#">Контакты</a></li>
                <li><a href="#">О сайте</a></li>
            </ul>
        </div>
    </div>
    <div id="content"><p>helo world</p></div>
</body>
</html>

```



Несмотря на то, что панель навигации определена в блоке с id header, визуально она явно не находится в элементе head. Ну и кроме того, здесь также можно увидеть, что появляются непонятные отступы, а следующий после хедера блок залезает на меню.

Проблема отступов заключается в том, что браузер по умолчанию определяет для различных элементов встроенные стили. Поэтому может немного сбивать с толку, как и где эти стили определены, почему они применяются. Нередко для решения этой проблемы разработчики просто сбрасывают некоторые наиболее значимые стили для большинства элементов:

```
html, body, div, span, h1, h2, h3, h4, h5, h6, p, a, img, dl, dt, dd,
ol, ul, li, form, table, caption, tr, th, td, article, aside, footer, header{

    margin: 0;
    padding: 0;
    border: 0;
    font-size: 100%;
    vertical-align: baseline;
}
```

Другая проблема - наложение элемента div с основным контентом на плавающий блок навигационной панели решается довольно просто - установкой для этого элемента div следующего стиля:

```
clear: both;
```

Более сложной является проблема с выпадением плавающих элементов меню из границ блока-контейнера. Здесь есть два возможных варианта Решения. Первое решение состоит в добавлении к элементу, который представляет панель навигации, следующего стиля:

```
ul:after {
    content: " ";
    display: table;
    clear: both;
}
```

Второе решение состоит в том, чтобы сделать сам блок панели навигации плавающим:

```
#nav{
    background-color: #f4f4f4;
    border-top: 1px solid #ccc;
    border-bottom: 1px solid #ccc;

    float:left;
    width: 100%;
    clear: both;
}
```

Итак, с учетом выше сказанного изменим стили для веб-страницы (код html остается прежним):


```

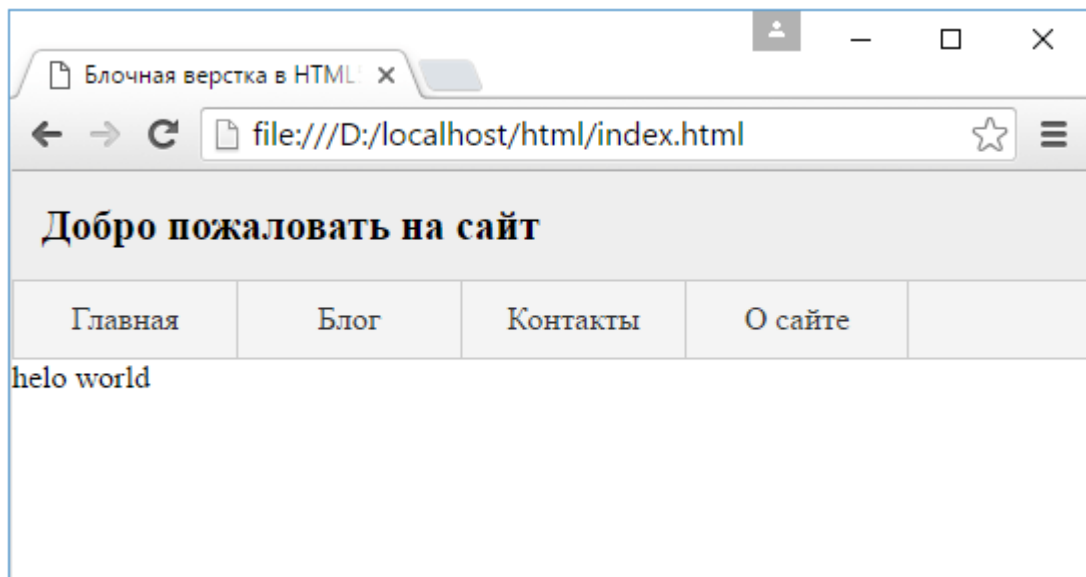
*{
    box-sizing: border-box;
}
html, body, div, span, h1, h2, h3, h4, h5, h6, p, a, img, dl, dt, dd,
ol, ul, li, form, table, caption, tr, th, td, article, aside, footer, header{

    margin: 0;
    padding: 0;
    border: 0;
    font-size: 100%;
    vertical-align: baseline;
}
#header{
    background-color: #eee;
}
#header h1{
    font-size: 1.3em;
    padding: 15px;
}
#nav{
    background-color: #f4f4f4;
    border-top: 1px solid #ccc;
    border-bottom: 1px solid #ccc;
}
#nav ul{
    margin-left: 0px;
    padding-left: 0px;
    list-style: none;
}
#nav li {
    float: left;
}
#nav ul a {
    display: block;
    width: 7em;
    padding: 10px;
    border-left: 1px solid #ccc;
    text-decoration: none;
    color: #333;
    text-align: center;
}
#nav ul li:last-child a {

    border-right: 1px solid #ccc;
}
#nav ul a:hover{
    background-color: #aaa;
    color: #f4f4f4;
}
#nav ul:after {
    content: " ";
    display: table;
    clear: both;
}
#content{
    clear: both;
}

```

И теперь веб-страница будет выглядеть иначе, собственно как и должна:



Создание простейшего макета

Используем полученные из прошлых тем сведения и создадим более менее осмысленный макет самой простейшей веб-страницы. Для начала определим базовую структуру веб-страницы:

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <link href="styles.css" rel="stylesheet">
    <title>Блочная верстка в HTML5</title>
  </head>
  <body>
    <div id="header">
      <h1>MySyte.com - Сайт о Lorem Ipsum</h1>
      <div id="nav">
        <ul>
          <li><a href="#">Главная</a></li>
          <li><a href="#">Блог</a></li>
          <li><a href="#">Форум</a></li>
          <li><a href="#">Контакты</a></li>
          <li><a href="#">О сайте</a></li>
        </ul>
      </div>
    </div>
    <div class="wrapper">
      <div id="sidebar1" class="aside">
        <h2>The standard Lorem Ipsum passage</h2>
        <p>"Lorem ipsum dolor sit amet, consectetur adipiscing elit,
sed do eiusmod tempor incididunt
ut labore et dolore magna aliqua..."</p>
      </div>
      <div id="sidebar2" class="aside">
        <h2>1914 translation by H. Rackham</h2>
        <p>It is a long established fact that a reader will be
distracted by the readable
```

```

        content of a page when looking at its layout.</p>
        <h3>Options</h3>
        <ul>
            <li>Item1</li>
            <li>Item2</li>
            <li>Item3</li>
        </ul>
    </div>
    <div id="article">
        <h2>What is Lorem Ipsum?</h2>
        <p>Lorem Ipsum is simply dummy text of the printing and
typesetting industry...</p>
        <p>Contrary to popular belief, Lorem Ipsum is not simply
random text. It has roots in a piece of
        classical Latin literature from 45 BC, making it over 2000
years old. Richard McClintock,
        a Latin professor at Hampden-Sydney College in
Virginia...</p>
    </div>
</div>
<div id="footer">
    <p>Contacts: admin@mysyte.com</p>
    <p>Copyright © MySyte.com, 2016</p>
</div>
</body>
</html>

```

В начале идет шапка сайта - блок с header, который содержит заголовок страницы и панель навигации. Далее идет блок wrapper, в котором два сайдбара и блок основного содержимого страницы. Сайдбары условно тоже содержат некоторое содержимое, но главное, что они определены до основного блока. И в самом низу небольшой футер.

В начале веб-страницы определено подключение файла `styles.css`, который будет стилизовать веб-страницу. Поэтому создадим в одном каталоге с веб-страницей файл `styles.css` и определим в нем следующее содержимое:

```

* {
    box-sizing: border-box;
}
html, body, div, span, h1, h2, h3, h4, h5, h6, p, a, ul, li{

    margin: 0;
    padding: 0;
    border: 0;
    font-size: 100%;
    vertical-align: baseline;
}
body {
    font-family: Verdana, Arial, sans-serif;
    background-color: #f7f7f7;
}
#header{
    background-color: #f4f4f4;
}
#header h1 {
    font-size: 24px;
    text-transform: uppercase;

```

```

        font-weight: bold;
        padding: 30px 30px 30px 10px;
        clear: both;
    }
    #nav {
        background-color: #eee;
        border-top: 1px solid #ccc;
        border-bottom: 1px solid #ccc;
    }
    #nav li {
        float: left;
        list-style: none;
    }
    #nav a {
        display: block;
        color: black;
        padding: 10px 25px;
        text-decoration: none;
        border-right: 1px solid #ccc;
    }
    #nav li:last-child a {
        border-right: none;
    }
    #nav a:hover {
        font-weight: bold;
    }
    #nav:after {
        content: " ";
        display: table;
        clear: both;
    }
    .wrapper{
        background-color: #f7f7f7;
    }
    .aside h2 {
        font-size: 0.95em;
        margin-top: 15px;
    }
    .aside h3 {
        font-size: 0.85em;
        margin-top: 10px;
    }
    .aside p, .aside li {
        font-size: .75em;
        margin-top: 10px;
    }
    .aside li{
        list-style-type: none;
    }
    #sidebar1 {
        float: left;
        width: 20%;
        padding: 0 10px 0 20px;
    }
    #sidebar2 {
        float: right;
        width: 20%;
        padding: 0 20px 0 10px;
    }
    #article{
        background-color: #fafafa;
        border-left: 1px solid #ccc;
        border-right: 1px solid #ccc;
    }

```

```

        margin-left: 20%;
        margin-right: 20%;
        padding: 15px;
        width: 60%;
    }
    #article:after{
        clear:both;
        display:table;
        content:'';
    }
    #article h2{
        font-size: 1.3em;
        margin-bottom:15px;
    }
    #article p{
        line-height: 150%;
        margin-bottom: 15px;
    }
    #footer{
        border-top: 1px solid #ccc;
        font-size: .8em;
        text-align: center;
        padding: 10px 10px 30px 10px;
    }
    #nav ul, #header h1, .wrapper, #footer p {
        max-width: 1200px;
        margin: 0 auto;
    }
    .wrapper, #nav, #header, #footer{
        min-width: 768px;
    }

```

Первые три стиля сбрасывают стилевые настройки по умолчанию для используемых нами элементов, а также устанавливают стиль элемента `body`.

Следующая пара стилей управляет отображением шапки (хедера) и заголовка страницы:

```

#header{
    background-color: #f4f4f4;
}
#header h1 {
    font-size: 24px;
    text-transform: uppercase;
    font-weight: bold;
    padding: 30px 30px 30px 10px;
    clear: both;
}

```

Следующий набор стилей управляет созданием горизонтальной панели навигации:

```

#nav {
    background-color: #eee;
    border-top: 1px solid #ccc;
    border-bottom: 1px solid #ccc;
}
#nav li {
    float: left;
    list-style: none;
}

```

```

#nav a {
    display: block;
    color: black;
    padding: 10px 25px;
    text-decoration: none;
    border-right: 1px solid #ccc;
}
#nav li:last-child a {
    border-right: none;
}
#nav a:hover {
    font-weight: bold;
}
#nav:after {
    content: " ";
    display: table;
    clear: both;
}

```

В одной из прошлых тем подробно обсуждалось создание горизонтальной панели навигации. В принципе здесь ничего нового не добавляется: для элементов `<li>` устанавливается обтекание (`float: left;`), благодаря чему они размещаются в ряд, а каждая ссылка делается блочным элементом (`display: block;`)

Далее идет настройка средней части страницы, в частности, сайдбаров:

```

.wrapper{
    background-color: #f7f7f7;
}
.aside h2 {
    font-size: 0.95em;
    margin-top: 15px;
}
.aside h3 {
    font-size: 0.85em;
    margin-top: 10px;
}
.aside p, .aside li {
    font-size: .75em;
    margin-top: 10px;
}
.aside li{
    list-style-type: none;
}
#sidebar1 {
    float: left;
    width: 20%;
    padding: 0 10px 0 20px;
}
#sidebar2 {
    float: right;
    width: 20%;
    padding: 0 20px 0 10px;
}

```

Стиль класса `wrapper` позволяет установить фоновый цвет для боковых панелей. Для каждого сайдбара определяется ширина в 20% от ширины страницы. Процентные значения позволяют автоматически подстраивать ширину блоков под ширину окна браузера при его расширении или сужении.

Далее следуют стили блока основного содержимого и футера:

```
#article{
  background-color: #fafafa;
  border-left: 1px solid #ccc;
  border-right: 1px solid #ccc;
  margin-left: 20%;
  margin-right: 20%;
  padding: 15px;
  width: 60%;
}
#article:after{
  clear:both;
  display:table;
  content:'';
}
#article h2{
  font-size: 1.3em;
  margin-bottom:15px;
}
#article p{
  line-height: 150%;
  margin-bottom: 15px;
}
#footer{
  border-top: 1px solid #ccc;
  font-size: .8em;
  text-align: center;
  padding: 10px 10px 30px 10px;
}
```

Поскольку боковые панели имеют ширину в 20% каждая, то для главного блока устанавливается ширина в 60% и отступы справа и слева в 20%.

И в конце идет пара довольно важных стилей:

```
#nav ul, #header h1, .wrapper, #footer p {
  max-width: 1200px;
  margin: 0 auto;
}
.wrapper, #nav, #header, #footer{
  min-width: 768px;
}
```

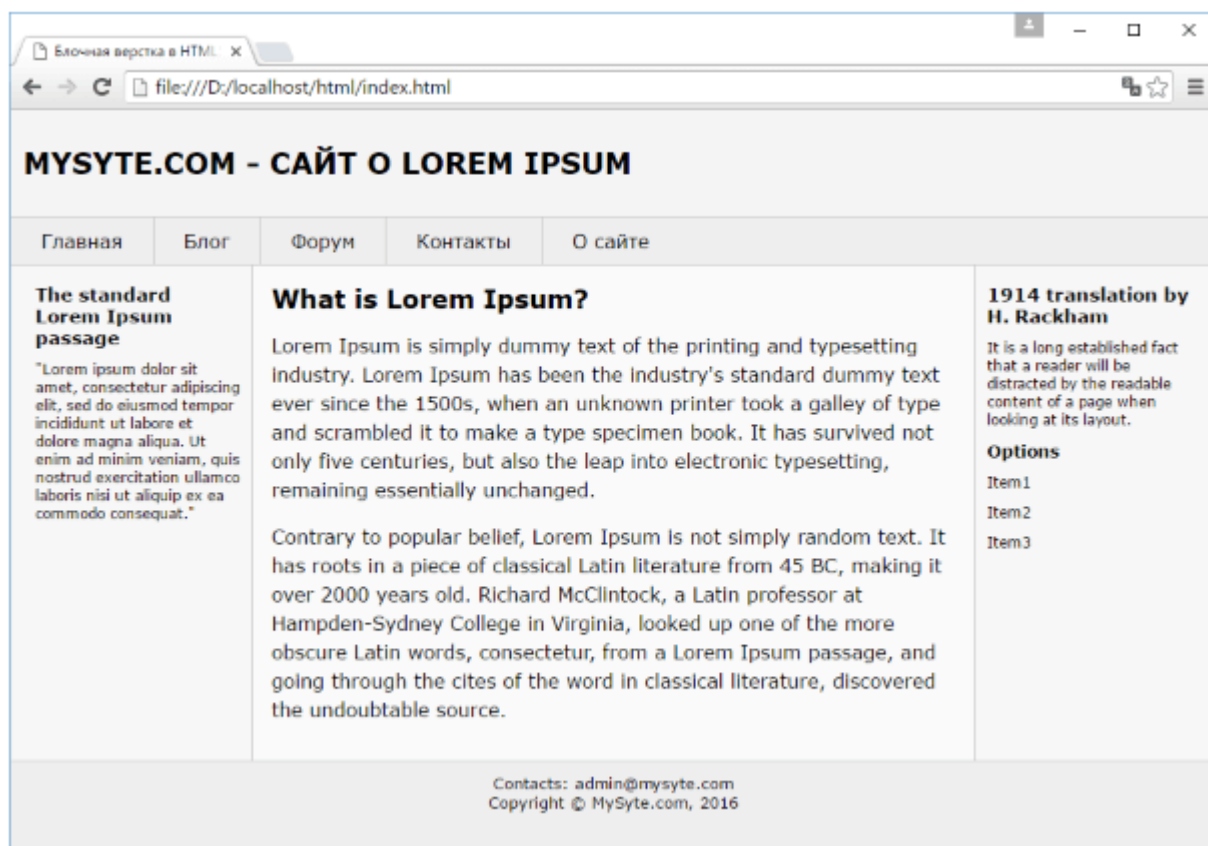
В начале для ряда селекторов определяется максимальная ширина в 1200 пикселей. Это значит, что основные элементы страницы не выйдут за пределы 1200 пикселей. А автоматический внешний отступ слева и справа позволит центрировать содержимое элементов. То есть при ширине браузера в 1400 пикселей эти элементы с шириной в 1200

пикселей будут размещаться как бы по середине, а справа и слева будут отступы шириной в $(1400-1200)/2 = 100$ пикселей.

Второй стиль позволят сделать фиксированную минимальную ширину для ряда элементов. То есть в итоге при сжатии окна браузера сайдбары и основной блок будут выглядеть более менее, а при сжатии окна менее 768 пикселей образуется полоса прокрутки.

Данная модель размеров не идеальна. И далее мы рассмотрим более гибкие и распространенные концепции на основе адаптивной верстки.

В итоге у нас получится следующий простенький макет:



Создание макета страницы и верстка

Позиционирование

CSS предоставляет возможности по позиционированию элемента, то есть мы можем поместить элемент в определенное место на странице

Основным свойством, которые управляют позиционированием в CSS, является свойство position. Это свойство может принимать одно из следующих значений:

- `static`: стандартное позиционирование элемента, значение по умолчанию
- `absolute`: элемент позиционируется относительно границ элемента-контейнера, если у того свойство `position` не равно `static`
- `relative`: элемент позиционируется относительно его позиции по умолчанию. Как правило, основная цель относительного позиционирования заключается не в том, чтобы переместить элемент, а в том, чтобы установить новую точку привязки для абсолютного позиционирования вложенных в него элементов
- `fixed`: элемент позиционируется относительно окна браузера, это позволяет создать фиксированные элементы, которые не меняют положения при прокрутке

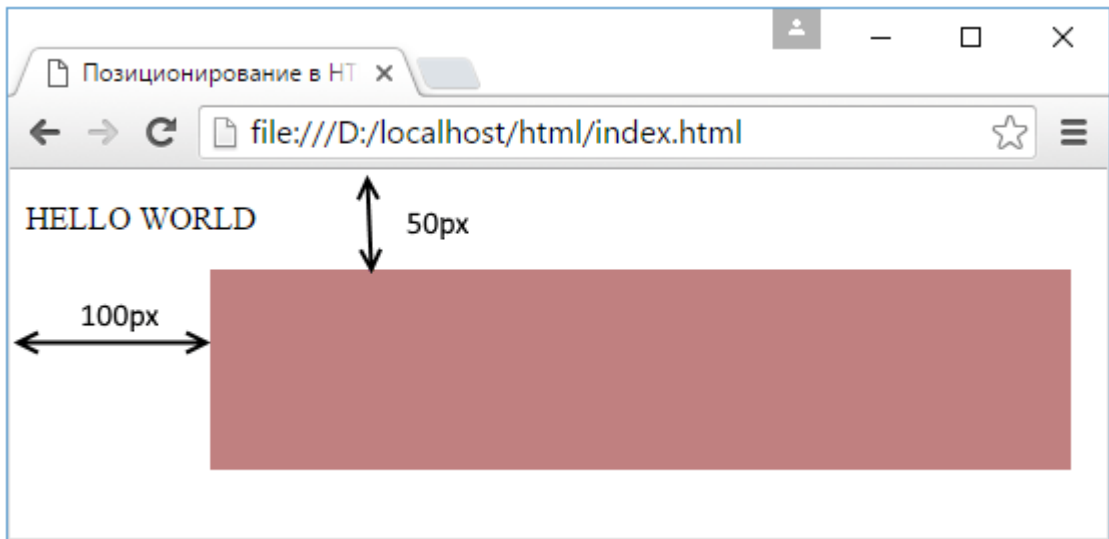
Не следует одновременно применять к элементу свойство `float` и любой тип позиционирования, кроме `static` (то есть тип по умолчанию).

Абсолютное позиционирование

Область просмотра браузера имеет верхний, нижний, правый и левый края. Для каждого из этих четырех краев есть соответствующее свойство CSS: `left` (отступ от края слева), `right` (отступ от края справа), `top` (отступ от края контейнера сверху) и `bottom` (отступ снизу). Значения этих свойств указываются в пикселях, `em` или процентах. Необязательно задавать значения для всех четырех сторон. Как правило, устанавливают только два значения - отступ от верхнего края `top` и отступ от левого края `left`.

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <title>Блочная верстка в HTML5</title>
    <style>
      .header {
        position: absolute;
        left: 100px;
        top: 50px;
        width: 430px;
        height: 100px;
        background-color: rgba(128, 0, 0, 0.5);
      }
    </style>
  </head>
  <body>
    <div class="header"></div>
    <p>HELLO WORLD</p>
  </body>
</html>
```

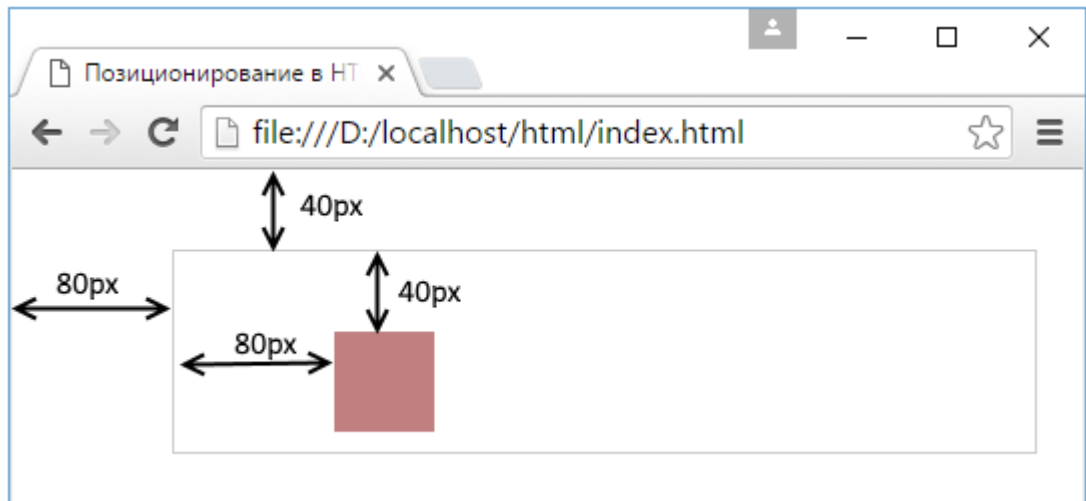
Здесь элемент `div` с абсолютным позиционированием будет находиться на 100 пикселей слева от границы области просмотра и на 50 снизу.



При этом не столь важно, что после этого элемента `div` идут какие-то другие элементы. Данный блок `div` в любом случае будет позиционироваться относительно границ области просмотра браузера.

Если элемент с абсолютным позиционированием располагается в другом контейнере, у которого в свою очередь значение свойства `position` не равно `static`, то элемент позиционируется относительно границ контейнера:

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <title>Позиционирование в HTML5</title>
    <style>
      .outer {
        position: absolute;
        left: 80px;
        top: 40px;
        width: 430px;
        height: 100px;
        border: 1px solid #ccc;
      }
      .inner{
        position: absolute;
        left: 80px;
        top: 40px;
        width: 50px;
        height: 50px;
        background-color: rgba(128, 0, 0, 0.5);
      }
    </style>
  </head>
  <body>
    <div class="outer">
      <div class="inner"></div>
    </div>
  </body>
</html>
```



Относительное позиционирование

Относительное позиционирование также задается с помощью значения `relative`. Для указания конкретной позиции, на которую сдвигается элемент, применяются те же свойства `top`, `left`, `right`, `bottom`:

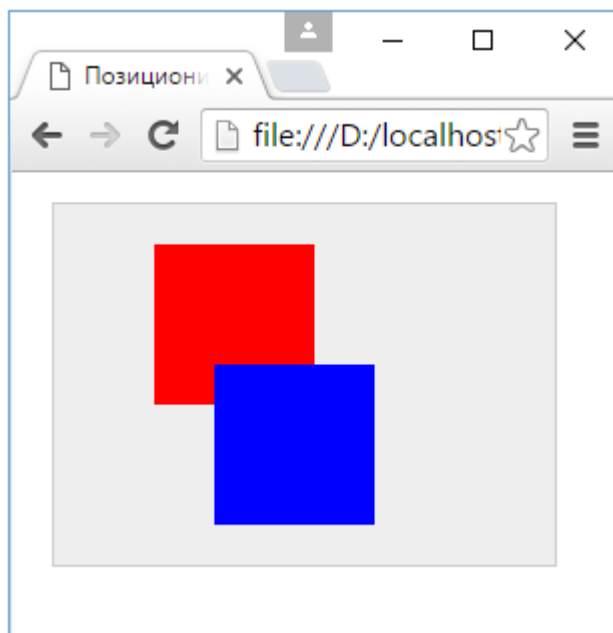
```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <title>Позиционирование в HTML5</title>
    <style>
      .outer {
        position: relative;
        left: 80px;
        top: 40px;
        width: 300px;
        height: 100px;
        border: 1px solid #ccc;
      }
      .inner{
        position: absolute;
        left: 80px;
        top: 40px;
        width: 50px;
        height: 50px;
        background-color: rgba(128, 0, 0, 0.5);
      }
    </style>
  </head>
  <body>
    <div class="outer"><div class="inner"></div></div>
  </body>
</html>
```

Свойство z-index

По умолчанию при совпадении у двух элементов границ, поверх другого отображается тот элемент, который определен в разметке html последним. Однако свойство z-index позволяет изменить порядок следования элементов при их наложении. В качестве значения свойство принимает число. Элементы с большим значением этого свойства будут отображаться поверх элементов с меньшим значением z-index.

Например:

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <title>Позиционирование в HTML5</title>
    <style>
      body{
        margin:0;
        padding:0;
      }
      .content{
        position: relative;
        top: 15px;
        left: 20px;
        width: 250px;
        height: 180px;
        background-color: #eee;
        border: 1px solid #ccc;
      }
      .redBlock{
        position: absolute;
        top: 20px;
        left: 50px;
        width: 80px;
        height: 80px;
        background-color: red;
      }
      .blueBlock{
        position: absolute;
        top: 80px;
        left: 80px;
        width: 80px;
        height: 80px;
        background-color: blue;
      }
    </style>
  </head>
  <body>
    <div class="content">
      <div class="redBlock"></div>
      <div class="blueBlock"></div>
    </div>
  </body>
</html>
```

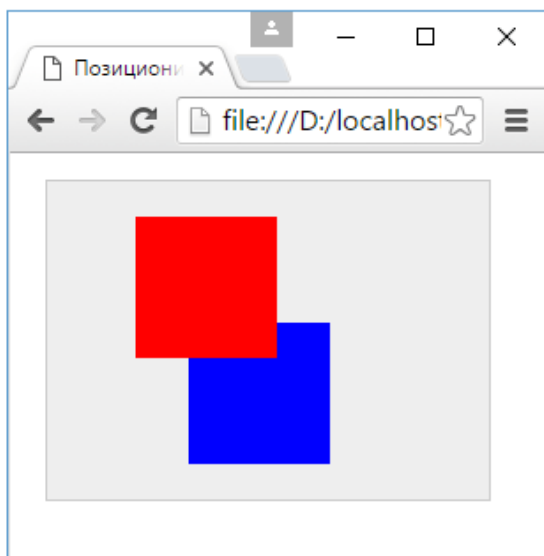


Теперь добавим к стилю блока redBlock новое правило:

```
.redBlock{  
    z-index: 100;  
  
    position: absolute;  
    top: 20px;  
    left: 50px;  
    width: 80px;  
    height: 80px;  
    background-color: red;  
}
```

Здесь z-index равен 100. Но это необязательно должно быть число 100. Так как у второго блока z-index не определен и фактически равен нулю, то для redBlock мы можем установить у свойства z-index любое значение больше нуля.

И теперь первый блок будет накладываться на второй, а не наоборот, как было в начале:



Фиксированное позиционирование

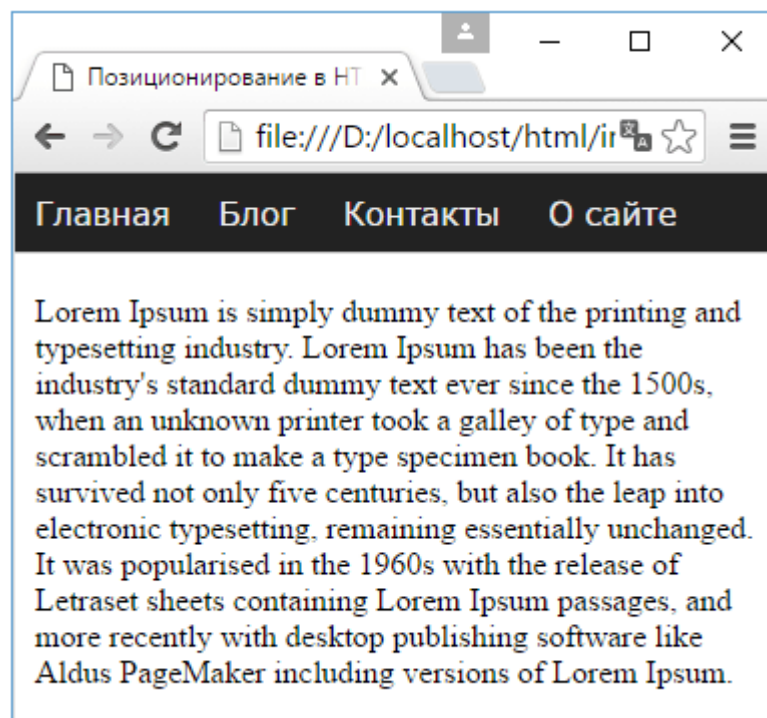
Фиксированное позиционирование является распространенным способом удерживать в области просмотра браузера некоторые элементы. Достаточно часто на различных сайтах можно увидеть фиксированную панель навигации, которая не изменяет своего положения вне зависимости от прокрутки.

Для фиксированного позиционирования у элементов нужно установить значение `fixed` для свойства `position`. После этого с помощью стандартных свойств `left`, `right`, `top` и `bottom` можно определить конкретную позицию фиксированного элемента.

Создадим к примеру фиксированную панель навигации:

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <title>Позиционирование в HTML5</title>
    <style>
      body{
        margin:0;
        padding:0;
      }
      .toolbar{
        position: fixed;
        top: 0;
        left: 0;
        right: 0;
        background-color: #222;
        border-bottom: 1px solid #ccc;
      }
      .toolbar a{
        color: #eee;
        display: inline-block;
        padding: 10px;
        text-decoration: none;
        font-family: Verdana;
      }
      .content{
        margin-top: 50px;
        padding: 10px;
      }
    </style>
  </head>
  <body>
    <div class="toolbar">
      <a href="#">Главная</a>
      <a href="#">Блог</a>
      <a href="#">Контакты</a>
      <a href="#">О сайте</a>
    </div>
    <div class="content">Lorem Ipsum is simply dummy text of the printing
and typesetting industry.
      Lorem Ipsum has been the industry....</div>
```

```
</body>  
</html>
```



Чтобы растянуть фиксированный блок от левой до правой границы страницы, устанавливаются три свойства:

```
top: 0;  
left: 0;  
right: 0;
```

Для нижележащего блока с основным содержанием фиксированный элемент фактически не существует разметке, так как блок с фиксированным, как и с абсолютным позиционированием не участвуют в стандартном потоке html. Поэтому по умолчанию оба блока будут накладываться друг на друга и размещаться в одной точке. И нам надо должным образом разместить блок содержимого относительно фиксированного блока, например, установив нужный отступ:

```
margin-top: 50px;
```

Фактически отступ идет от границ области просмотра браузера, поэтому высота отступа должна быть больше высоты фиксированного элемента.

Задание к лабораторной работе:

1. Выполнить примеры, приведенные в лабораторной работе: каждый пример выполняем в виде отдельной страницы с обязательным названием

выполняемого примера. Название должно отражать суть выполненного задания.

2. Сделать главную страницу к лабораторной работе, отражающую тему лабораторной работы и имеющую переходы на страницы из примеров, выполненные в задании 1.

3. Создать собственную страницу, используя блочную верстку. Для каждого блока определить цвет фона. Обязательно на странице примените вложенные плавающие блоки, продемонстрируйте работу свойства – display, использовать свойства inline-block, run-in. Выполнить сокрытие элемента. На странице создайте панель навигации.

4. Создать страницу, продемонстрировать умение работать с позиционированием элементов, обязательно использовать свойство z-index.